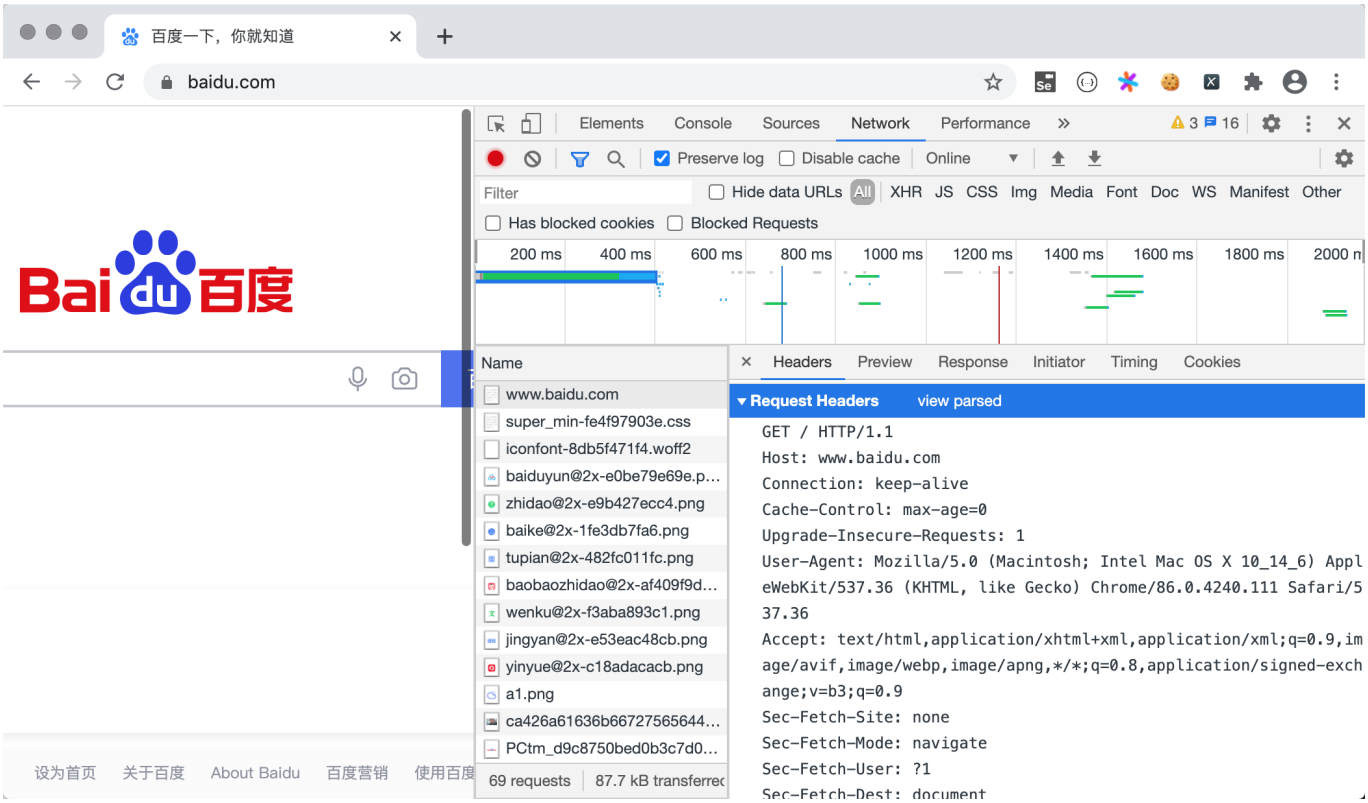
用Python获取网络数据

网络数据采集是 Python 语言非常擅长的领域，上节课我们讲到，实现网络数据采集的程序通常称之为网络爬虫或蜘蛛程序。即便是在大数据时代，数据对于中小企业来说仍然是硬伤和短板，有些数据需要通过开放或付费的数据接口来获得，其他的行业数据和竞对数据则必须要通过网络数据采集的方式来获得。不管使用哪种方式获取网络数据资源，Python 语言都是非常好的选择，因为 Python 的标准库和三方库都对网络数据采集提供了良好的支持。

requests库

要使用 Python 获取网络数据，我们推荐大家使用名为`requests` 的三方库，这个库我们在之前的课程中其实已经使用过了。按照官方网站的解释，`requests`是基于 Python 标准库进行了封装，简化了通过 HTTP 或 HTTPS 访问网络资源的操作。上课我们提到过，HTTP 是一个请求响应式的协议，当我们在浏览器中输入正确的 URL（通常也称为网址）并按下 Enter 键时，我们就向网络上的 Web 服务器发送了一个 HTTP 请求，服务器在收到请求后会给我们一个 HTTP 响应。在 Chrome 浏览器中的菜单中打开“开发者工具”切换到“Network”选项卡就能够查看 HTTP 请求和响应到底是什么样子的，如下图所示。



通过`requests`库，我们可以让 Python 程序向浏览器一样向 Web 服务器发起请求，并接收服务器返回的响应，从响应中我们就可以提取出想要的数据。浏览器呈现给我们的网页是用 HTML 编写的，浏览器相当于是 HTML 的解释器环境，我们看到的网页中的内容都包含在 HTML 的标签中。在获取到 HTML 代码后，就可以从标签的属性或标签体中提取内容。下面例子演示了如何获取网页 HTML 代码，我们通过`requests`库的`get`函数，获取了搜狐首页的代码。

```
import requests

resp = requests.get('https://www.sohu.com/')
if resp.status_code == 200:
    print(resp.text)
```

> **说明**：上面代码中的变量resp是一个Response对象（requests库封装的类型），通过该对象的status_code属性可以获取响应状态码，而该对象的text属性可以帮我们获取到页面的 HTML 代码。

由于Response对象的text是一个字符串，所以我们可以利用之前讲过的正则表达式的知识，从页面的 HTML 代码中提取新闻的标题和链接，代码如下所示。

```
import re

import requests

pattern = re.compile(r'<a.*?href="(.*?)".*?title="(.*?)".*?>')
resp = requests.get('https://www.sohu.com/')
if resp.status_code == 200:
    all_matches = pattern.findall(resp.text)
    for href, title in all_matches:
        print(href)
        print(title)
```

除了文本内容，我们也可以使用requests库通过 URL 获取二进制资源。下面的例子演示了如何获取百度 Logo 并保存到名为baidu.png的本地文件中。可以在百度的首页上右键点击百度Logo，并通过“复制图片地址”菜单项获取图片的 URL。

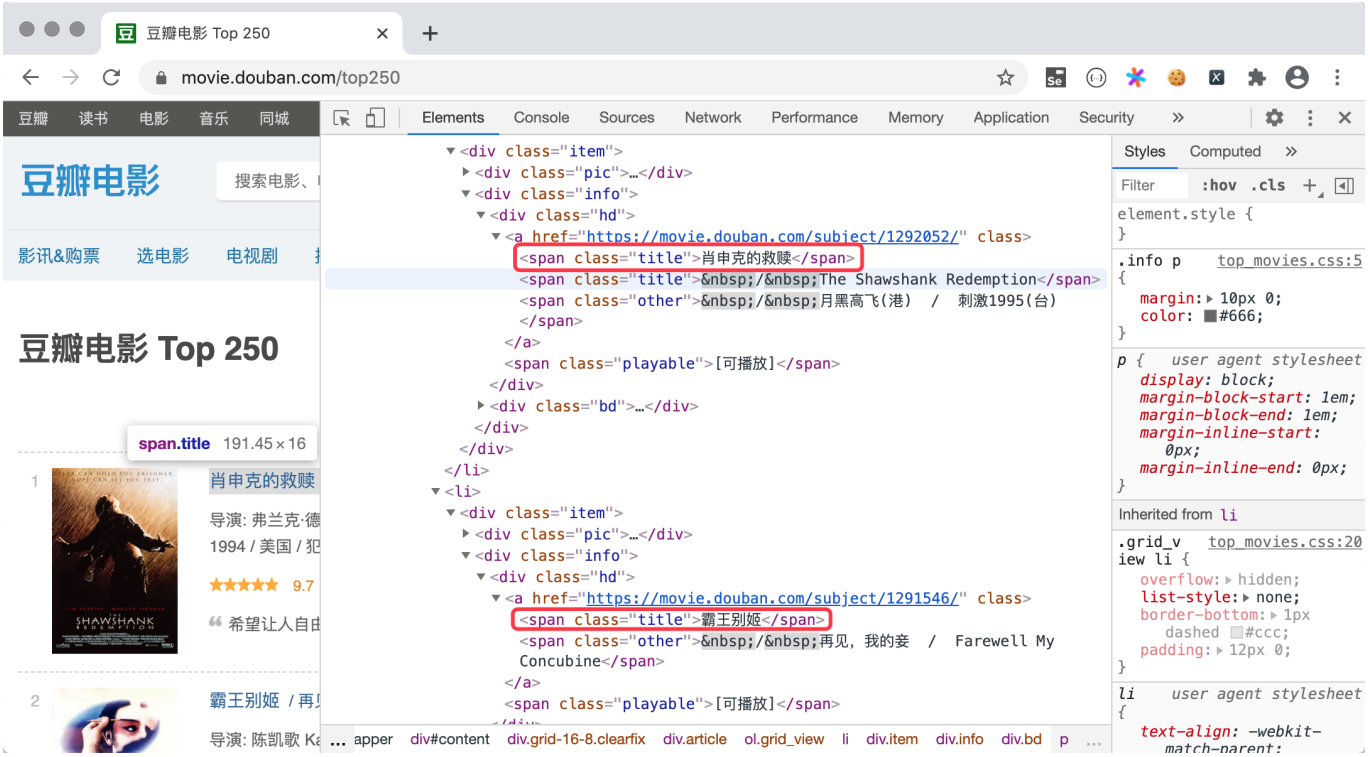
```
import requests

resp =
requests.get('https://www.baidu.com/img/PCtm_d9c8750bed0b3c7d089fa7d55720d6cf.png'
)
with open('baidu.png', 'wb') as file:
    file.write(resp.content)
```

> **说明**：Response对象的content属性可以获得服务器响应的二进制数据。

requests库非常好用而且功能上也比较强大和完整，具体的内容我们在使用的过程中为大家一点点剖析。想解锁关于requests库更多的知识，可以阅读它的官方文档。

编写爬虫代码

接下来，我们以“豆瓣电影”为例，为大家讲解如何编写爬虫代码。按照上面提供的方法，我们先使用requests获取到网页的HTML代码，然后将整个代码看成一个长字符串，这样我们就可以使用正则表达式的捕获组从字符串提取我们需要的内容。下面的代码演示了如何从豆瓣电影获取排前250名的电影的名称。豆瓣电影Top250的页面结构和对应代码如下图所示，可以看出，每页共展示了25部电影，如果要获取到 Top250 数据，我们共需要访问10个页面，对应的地址是https://movie.douban.com/top250?start=xxx，这里的xxx如果为0就是第一页，如果xxx的值是100，那么我们可以访问到第五页。为了代码简单易读，我们只获取电影的标题和评分。




```python
import random
import re
import time

import requests

for page in range(1, 11):
    resp = requests.get(
        url=f'https://movie.douban.com/top250?start={(page - 1) * 25}',
        # 如果不设置HTTP请求头中的User-Agent，豆瓣会检测出不是浏览器而阻止我们的请求。
        # 通过get函数的headers参数设置User-Agent的值，具体的值可以在浏览器的开发者工具查看到。
        # 用爬虫访问大部分网站时，将爬虫伪装成来自浏览器的请求都是非常重要的一步。
        headers={'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/92.0.4515.159 Safari/537.36'}
    )
    # 通过正则表达式获取class属性为title且标签体不以&开头的span标签并用捕获组提取标签内容
    pattern1 = re.compile(r'<span class="title">([^&]*?)</span>')
    titles = pattern1.findall(resp.text)
    # 通过正则表达式获取class属性为rating_num的span标签并用捕获组提取标签内容
    pattern2 = re.compile(r'<span class="rating_num".*?>(.*?)</span>')
    ranks = pattern2.findall(resp.text)
    # 使用zip压缩两个列表，循环遍历所有的电影标题和评分
    for title, rank in zip(titles, ranks):
        print(title, rank)
    # 随机休眠1-5秒，避免爬取页面过于频繁
    time.sleep(random.random() * 4 + 1)
```

> **说明**：通过分析豆瓣网的robots协议，我们发现豆瓣网并不拒绝百度爬虫获取它的数据，因此我们也可以将爬虫伪装成百度的爬虫，将`get`函数的`headers`参数修改为：`headers={'User-Agent': 'BaiduSpider'}`。